

SafePath AI

Machine Learning Component

Study Guide & Viva Preparation

XGBoost risk-prediction model: architecture, dataset, features, training metrics, integration, and 20 viva questions with answers.

Final-year engineering project

Table of Contents

Part 1	What is this model?
Part 2	The dataset
Part 3	Features
Part 4	Training and testing statistics
Part 5	Why XGBoost?
Part 6	How the model works in the project
Part 7	Twenty viva questions and answers
Closing	Tips for the viva

Part 1 — What is this model?

The project uses **XGBoost** — short for **eXtreme Gradient Boosting**. It is a supervised machine-learning algorithm based on **gradient-boosted decision trees**.

Two XGBoost models are deployed together, sharing the same features and training data:

	Regressor	Classifier
File	ml-service/risk_model.json	ml-service/risk_classifier.json
Class	XGBRegressor	XGBClassifier
Output	Continuous 0-100 risk score	LOW / MEDIUM / HIGH / CRITICAL
Problem type	Regression	Multi-class classification
Loss function	Mean Absolute Error (MAE)	Multi-class log-loss (softprob)
Trees (max)	500 (early-stopped)	500 (early-stopped)
Tree depth	6	6
Learning rate	0.05	0.05
Row subsample	0.85	0.85
Col subsample	0.85	0.85

Conceptually

An XGBoost ensemble is a sum of many shallow decision trees, where each tree corrects the errors of the trees before it. For the regressor:

$$\text{prediction}(x) = \text{tree}_1(x) + \text{tree}_2(x) + \text{tree}_3(x) + \dots + \text{tree}_N(x)$$

Each tree is small (depth 6 = at most 64 leaves) and contributes a small correction (learning rate 0.05). With up to 500 such trees, the model captures complex non-linear relationships that a single tree cannot.

Part 2 — The dataset

Three datasets are combined, not one.

2.1 Primary training data — risk_zones.csv

- Size: 219,318 rows x 10 columns
- Source: generated by generate_synthetic_data.py from real NCRB crime statistics. It scatters synthetic incident points within each district proportional to real reported crime counts, then snaps them to a 0.005 degree grid.
- Columns: latitude, longitude, gridResolution, riskScore, riskLevel, crimeDensity, nightCrimeRate, severityScore, crimeCount, year.
- Role: contains both the input features AND both target variables (riskScore for regression, riskLevel for classification).

2.2 District ground truth — NCRB 2014

- File: `Datasets/Crimes` in `india/crime/42_District_wise_crimes_committed_against_women_2014.csv`
- Source: National Crime Records Bureau (Ministry of Home Affairs, Government of India).
- Size: 801 districts after dropping summary/total rows.
- Columns used: state, district, plus 7 crime totals — Rape, Kidnapping & Abduction, Dowry Deaths, Assault on Women, Insult to Modesty, Cruelty by Husband, Total Crimes against Women.
- Role: provides real ground-truth context about each district's crime profile, used as model features.

2.3 District geolocation index — synthetic_crime_data.csv

- Size: 53,660 rows x 10 columns.
- Role: builds an index of 1,093 unique (state, district) centroids by averaging latitude / longitude. At inference time, the API uses this to find the nearest district to a queried lat/long so it can join the NCRB features.

2.4 Train / test split

Setting	Value
Method	scikit-learn train_test_split
Test size	20%
Random state	42 (reproducibility)
Stratification	By classification label (LOW/MEDIUM/HIGH/CRITICAL)
Train rows	175,454
Test rows	43,864
Cross-validation	Not used — single 80/20 split is sufficient at this scale

Part 3 — Features

3.1 The 15 features that were selected

Defined in `train_risk_model.py` at the `FEATURE_COLS` list.

#	Feature	Source	Type
1	latitude	Request	Spatial
2	longitude	Request	Spatial
3	year_filled	Constant (2014)	Temporal
4	crimeDensity	Postgres crime_records	Local
5	crimeCount	Postgres crime_records	Local
6	nightCrimeRate	Postgres crime_records	Local
7	log_ncrb_rape	NCRB district CSV	District context
8	log_ncrb_kidnap	NCRB district CSV	District context
9	log_ncrb_dowry	NCRB district CSV	District context
10	log_ncrb_assault	NCRB district CSV	District context
11	log_ncrb_insult	NCRB district CSV	District context
12	log_ncrb_cruelty	NCRB district CSV	District context
13	log_ncrb_total	NCRB district CSV	District context
14	frac_rape	Rape / district total	District composition
15	frac_kidnap, frac_dowry, frac_assault, frac_cruelty, frac_insult, frac_total	Composition fractions	District composition

3.2 What was deliberately excluded — and why

severityScore was dropped even though it was available in `risk_zones.csv`. The data generator computed the target by $riskScore \approx 1.5 \times severityScore \times crimeCount$. Including `severityScore` would let the model memorise the formula — a **tautology** — and report an artificial R^2 near 1.0. Excluding it forces the model to genuinely infer risk from independent contextual features.

This is a standard concept in ML called **target leakage**: a feature that is a direct function of the target should never be used as input.

3.3 Feature importance (regressor)

Feature	Importance
crimeDensity	0.711
crimeCount	0.070
nightCrimeRate	0.062
log_ncrb_cruelty	0.050
log_ncrb_assault	0.016
latitude	0.015
log_ncrb_total	0.012
frac_dowry	0.008

Local crime density dominates (~71%), with NCRB district context contributing ~14% combined and pure spatial coordinates contributing ~3%. This matches intuition — what is happening near you matters more than where you are on the map.

Part 4 — Training and testing statistics

4.1 Held-out test set (43,864 rows)

Regressor

Metric	Value	Meaning
MAE	6.42	Average predicted risk is within +/- 6.4 points of the true score
R ²	0.936	Model explains 93.6% of the variance in the test set

Classifier

Metric	Value
Overall accuracy	0.856
Weighted F1	0.859
Macro F1	0.799

Per-class breakdown

Class	Precision	Recall	F1-score	Support
LOW	0.965	0.900	0.931	20,267
MEDIUM	0.657	0.755	0.703	8,339
HIGH	0.660	0.659	0.659	3,393
CRITICAL	0.899	0.909	0.904	11,865

How to read this: LOW and CRITICAL (the extreme classes) achieve ~90% F1 — the model is confident about clearly safe and clearly dangerous areas. MEDIUM and HIGH (the middle classes) are harder — ~66-70% F1 — because the boundary between them is fuzzy. This is normal for ordinal multi-class problems with arbitrary cutoffs.

4.2 Training settings that matter

- **Early stopping:** training halts after 25 rounds with no improvement on the held-out test set. Prevents overfitting and saves time.
- **tree_method='hist':** uses histogram-based tree building. Much faster on CPU than the default exact method for large datasets.
- **subsample=0.85:** each tree sees a random 85% of rows. Reduces overfitting.
- **colsample_bytree=0.85:** each tree sees a random 85% of features. Same purpose.
- **learning_rate=0.05:** small step size means more trees needed but better generalisation.
- **max_depth=6:** shallow trees prevent overfitting individual training rows.
- **Training time:** ~30 seconds on a laptop CPU.

Part 5 — Why XGBoost?

A defensible answer when asked "why this model and not X?"

Reason	Explanation
Tabular data	Our data is structured. XGBoost beats neural nets on tabular with <1M rows.
Mixed feature scales	Trees don't need scaling; neural nets do.
Non-linearity	Tree splits naturally model feature interactions.
Fast training	30 seconds for 175k rows.
Fast inference	<5 ms per prediction. Suitable for real-time API.
Feature importance	Transparent for a safety app where explanations matter.
Robust to missing data	XGBoost handles NaN natively.
Robust to outliers	A single extreme value cannot destroy the model.
State of the art	Wins majority of Kaggle tabular competitions since 2016.
Small model file	~600 KB. Easy to deploy and version-control.

Alternatives we rejected

Alternative	Why rejected
Linear / Logistic Regression	Cannot capture interactions like night + dense crime + specific district.
Random Forest	Comparable accuracy but slower; boosting beats bagging on most tabular benchmarks.
Neural Network (MLP)	Needs feature scaling, more data, harder to tune, ~10x slower training.
K-Nearest Neighbors	Inference $O(N)$ per query — too slow with 175k training rows.
Support Vector Machine	Does not scale beyond ~10k rows efficiently.

Part 6 — How the model works in the project

6.1 At training time

```
risk_zones.csv          NCRB 2014 (801 districts)    synthetic_crime_data.csv
    |                    |                          |
    | build district centroid index <-----+      |
    v                    v                          v
attach_district() --> nearest centroid per row -> (state, district)
    |
    v
build_features() --> join NCRB -> 15-column feature matrix
    |
    v
train_test_split --> 175k train + 44k test (stratified)
    |
    +--> XGBRegressor.fit() --> risk_model.json
    +--> XGBClassifier.fit() --> risk_classifier.json
                                   district_centroids.csv
                                   district_ncrb.csv
                                   risk_model_meta.json
```

6.2 At inference time

```
HTTP POST /predict-risk {lat, lng}
    |
    v
ml-service/main.py predict_risk()
    |
    +-- Postgres query: crime_records within +-0.003 deg
    |     rows = [(severity, timeOfDay, caseCount), ...]
    |
    +-- spatial features:
    |     crimeDensity, crimeCount, nightCrimeRate
    |
    +-- nearest district from district_centroids.csv
    +-- NCRB features from district_ncrb.csv
    |
    +-- build 15-column feature row
    |
    +-- XGBRegressor.predict() -> risk_score 0-100
    +-- XGBClassifier.predict() -> LOW/MEDIUM/HIGH/CRITICAL
    |
    +-- return JSON {risk_score, risk_level, district, state, source}
```

Total round-trip ~20 ms. Model inference itself is <5 ms; the rest is Postgres query plus network.

6.3 Where predictions surface in the app

Honest disclosure: the model is reachable at <http://localhost:8000/predict-risk> but the React UI does not call it yet. Two Next.js proxy routes exist (`app/api/risk/score/route.ts` and `app/api/risk/bulk/route.ts`) but they have a broken env var (`ML_API_URL` vs `ML_SERVICE_URL`) and no component invokes them. The `SafeRouteMap` reads risk directly from the Postgres `riskZone` table, bypassing the model. This is a known integration gap — see Q20 below for the viva-safe answer.

Part 7 — Twenty viva questions and answers

Q1. What is XGBoost in one line?

XGBoost is a regularised, optimised implementation of gradient-boosted decision trees for supervised learning on structured data.

Q2. What is gradient boosting?

Gradient boosting is an ensemble technique that builds models sequentially. Each new model is trained to predict the residual errors (gradients of the loss) of the combined previous models. The final prediction is a weighted sum of all individual outputs. The 'gradient' refers to using the gradient of the loss function to decide what direction each new tree should correct.

Q3. Why did you use XGBoost instead of a deep neural network?

Three reasons. First, our data is tabular with only 15 features and ~219k rows; neural nets shine on unstructured data and large datasets, while gradient-boosted trees consistently win on tabular under 1M rows. Second, no feature scaling is needed because trees split on raw values regardless of scale. Third, XGBoost trains in 30 seconds, gives feature importance, and predicts in under 5 ms — a neural network would be slower, harder to tune, and less interpretable.

Q4. What is the difference between bagging and boosting?

Bagging (e.g. Random Forest) trains many models in parallel on random subsets and averages their predictions — it reduces variance. Boosting (e.g. XGBoost) trains models sequentially, where each new model focuses on examples the previous models got wrong — it reduces both bias and variance and usually achieves higher accuracy.

Q5. What is your target variable?

Two targets are used. The regression target is riskScore — a continuous value from 0 to 100 representing how dangerous a location is. The classification target is riskLevel — one of four ordinal labels LOW, MEDIUM, HIGH, CRITICAL — derived from riskScore by binning.

Q6. Why train two models instead of one?

The regressor gives a precise numeric score useful for ranking and visualisation (heatmap intensities). The classifier gives a categorical label useful for UI badges and traffic-light displays. Both train on the same features but optimise different losses and produce different output types. There is no extra inference cost since both run in under 5 ms.

Q7. What evaluation metrics did you use and why?

For the regressor we use **MAE** (6.42) because it is in the same units as the target and is robust to outliers, and **R²** (0.936) as a unit-free measure of fit. For the classifier we use **accuracy** (85.6%) plus **per-class precision, recall, and F1** because accuracy alone is misleading on imbalanced classes — our LOW class has 20k samples while HIGH has only 3.4k.

Q8. Your classifier is 85% accurate but only 66% F1 on the HIGH class. Why?

HIGH is sandwiched between MEDIUM and CRITICAL with arbitrary cutoffs at riskScore = 60 and 80. A prediction of 78 vs 82 is a real disagreement to the model but practically identical. The model confuses HIGH with MEDIUM (just below) and CRITICAL (just above), while the extreme classes LOW and CRITICAL achieve ~90% F1. This is normal for ordinal classification with hard cutoffs.

Q9. What is overfitting and how did you prevent it?

Overfitting occurs when a model memorises training data but fails on new data. Five techniques were used: (1) early stopping after 25 rounds with no improvement; (2) shallow trees (max_depth=6); (3) low learning rate (0.05); (4) row subsampling at 0.85; (5) column subsampling at 0.85. No overfitting was observed: R^2 was measured on the 20% held-out test set the model never saw.

Q10. What is target leakage and did you avoid it?

Target leakage is when a feature contains information that would not be available at prediction time or is a direct function of the target. The generator computed riskScore from severityScore. We **deliberately excluded severityScore** from the features. Including it would have produced an inflated R^2 near 1.0 that does not generalise. The remaining 0.936 R^2 reflects genuine learning.

Q11. Explain the train/test split you used.

scikit-learn train_test_split with test_size=0.2, random_state=42 for reproducibility, and stratify=y_clf so the test set has the same class distribution as training. K-fold cross-validation was not used because 175k training rows are sufficient for a single split to be statistically reliable, and CV would have multiplied training time by 5x without meaningfully changing confidence in the metrics.

Q12. What is feature importance and what does yours tell you?

Feature importance ranks how much each feature contributed to the model. XGBoost computes it as the average gain in the loss function when the feature is used to split a node. Our top features are crimeDensity (71%), crimeCount (7%), nightCrimeRate (6%), and log_ncrb_cruelty (5%). The model is interpretable: places with more nearby crimes, more night-time crimes, and in districts reporting more cruelty are predicted to be more dangerous. It is not a black box.

Q13. Why did you log-transform the NCRB features?

NCRB district totals span four orders of magnitude — some districts report ~10 crimes per year, others ~10,000. A linear scale would cluster most values at the low end, making tree splits ineffective. $\log(1 + x)$ compresses this range, making differences between low-crime districts as visible to the model as differences between high-crime ones. The +1 inside the log avoids $\log(0) = -\infty$ for zero counts.

Q14. How does your model handle a request for a district it never saw?

Two layers of robustness. First, the nearest-district lookup always returns some district — even a point in a data-sparse region falls back to the geographically closest known district. Second, if that district is not in the NCRB lookup, the NCRB features are zeroed (treated as 'no district evidence') and the model still produces a prediction from the spatial features and coordinates. The model degrades gracefully rather than crashing on novel inputs.

Q15. What is the inference latency and is it suitable for production?

Per request: ~10-15 ms Postgres query, <1 ms nearest-district lookup, <1 ms NCRB join, <5 ms combined XGBoost predictions — total ~20 ms end-to-end. This is suitable for real-time interactive use (under 100 ms is the human-perception threshold for 'instant'). For batch use cases like pre-computing a city grid, we can predict thousands of points per second using a single batched predict() call.

Q16. Why join NCRB district-level data when you already have per-location features?

To prevent the model from learning a purely local pattern that fails on new locations. Local features only see incidents within 300 m of the query point. The NCRB district context provides a macro signal — 'this point is in a district where total annual crimes against women are 5,000 with 30% being cruelty'. The combination of micro (local) and macro (district) features is why we hit 0.936 R² and produce sensible predictions even in sparse-data areas.

Q17. What data preprocessing did you do?

Five steps: (1) removed rows with missing latitude / longitude / riskScore / riskLevel; (2) stripped and upper-cased state and district names for consistent joining; (3) dropped NCRB summary rows containing 'TOTAL'; (4) log-transformed NCRB count features to compress the dynamic range; (5) filled NaN with 0 for districts unmatched in NCRB. Feature scaling was **not** applied because XGBoost is invariant to monotonic transformations — it only uses split thresholds.

Q18. How would you improve this model further?

Five concrete next steps. (1) Add temporal features: hour-of-day, day-of-week, month — currently year_filled is constant. (2) Train on multiple years of NCRB data (2001-2014) to capture trend changes. (3) Add population density — risk-per-person matters more than absolute count. (4) Hyperparameter tuning with Optuna or GridSearchCV to squeeze 1-2% more accuracy. (5) SHAP analysis for per-prediction explanations powering a 'why is this CRITICAL?' tooltip.

Q19. What are the limitations and ethical considerations?

Limitations: training labels come from synthetic data, not real incident reports; NCRB features are static (2014 only); district boundaries are coarse so a single district can contain both safe and dangerous neighbourhoods. **Ethical concerns:** feedback loops (avoided areas under-report, shifting their measured risk); stigmatisation (labelling districts has social and economic consequences); privacy (location data must not be logged in a way that reveals travel patterns). The model should be **decision support, not decision authority** — emergency responses must never depend on a model prediction alone.

Q20. Where exactly is XGBoost called at runtime, and what proof can you show?

XGBoost lives in ml-service/main.py and is served via FastAPI at POST /predict-risk. The model is loaded once at startup by the _ModelBundle class. Each request triggers a Postgres spatial query, builds a 15-feature row, and calls XGBRegressor.predict() and XGBClassifier.predict().

Live demo: curl -X POST http://localhost:8000/predict-risk -H "Content-Type: application/json" -d '{"lat":28.6139,"lng":77.2090}'. The response includes "source": "xgboost" which confirms the prediction came from the model. The FastAPI log shows 'Loaded XGBoost model (MAE=6.42, R2=0.936)' on first request.

Honest disclosure if pressed: the React frontend does not currently call this endpoint — the SafeRouteMap reads risk from a Postgres table seeded with formula-generated values. Connecting the React UI to call /predict-risk is the next integration step. The model is fully built, trained, validated, and serving — it just is not yet driving the user-facing screens.

Closing — Tips for the viva

- **Do not memorise numbers — understand the meaning.** If asked "what's your R^2 ?" and you blank, you can derive it: "It is how much of the variance the model explains. Mine is high, around 0.93, which means the model fits the test data well without overfitting."
- **Defend your choices, do not apologise for them.** "I chose XGBoost because..." beats "I just used XGBoost because the tutorial did."
- **Own the limitations.** Examiners trust students who say "this is what we couldn't do, and here is why" more than students who claim everything is perfect.
- **Open the FastAPI docs page** (<http://localhost:8000/docs>) before the viva starts. If you need to demo a prediction live, you can do it through the interactive Swagger UI without typing curl.
- **One sentence per file:** be able to say what each file does. `train_risk_model.py` trains. `main.py` serves. `risk_model.json` is the trained weights. `district_centroids.csv` is the lat/lng lookup. `district_ncrb.csv` is the feature lookup.

Good luck.